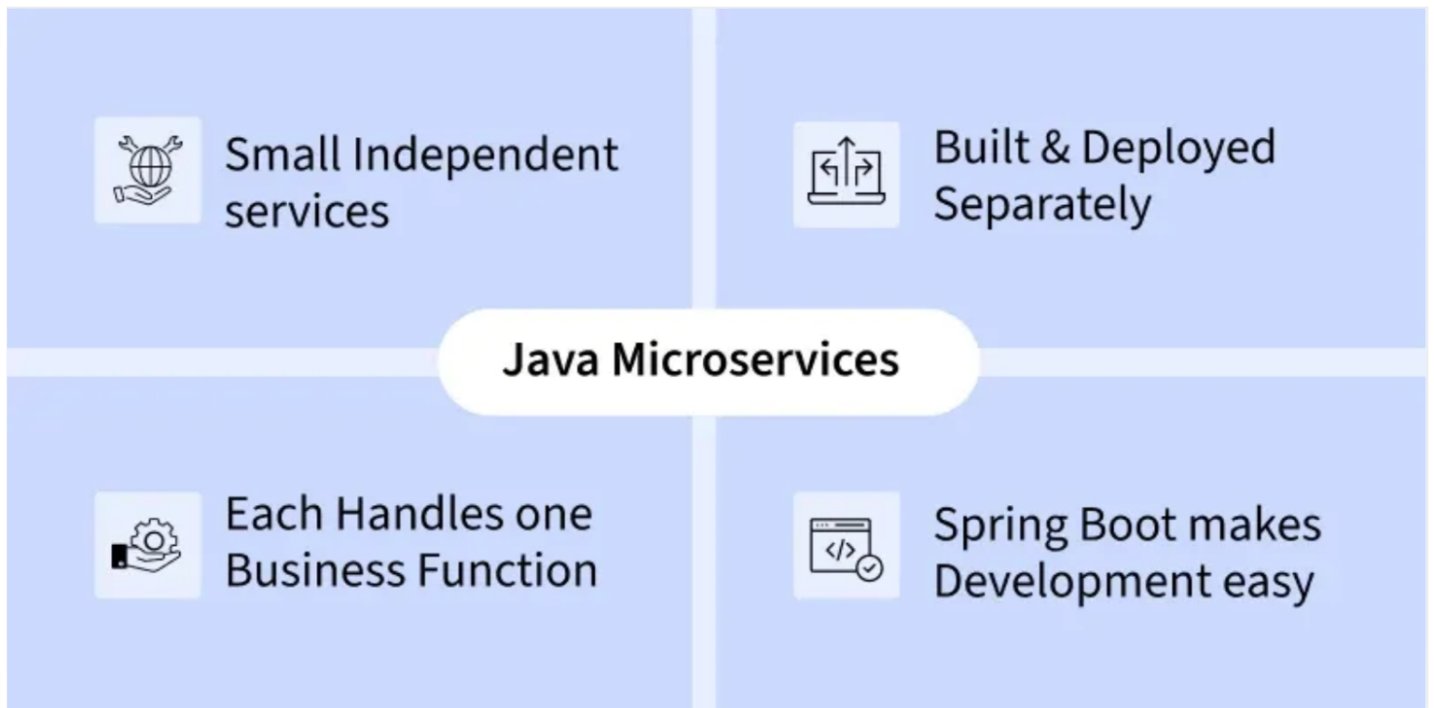


What are Microservices

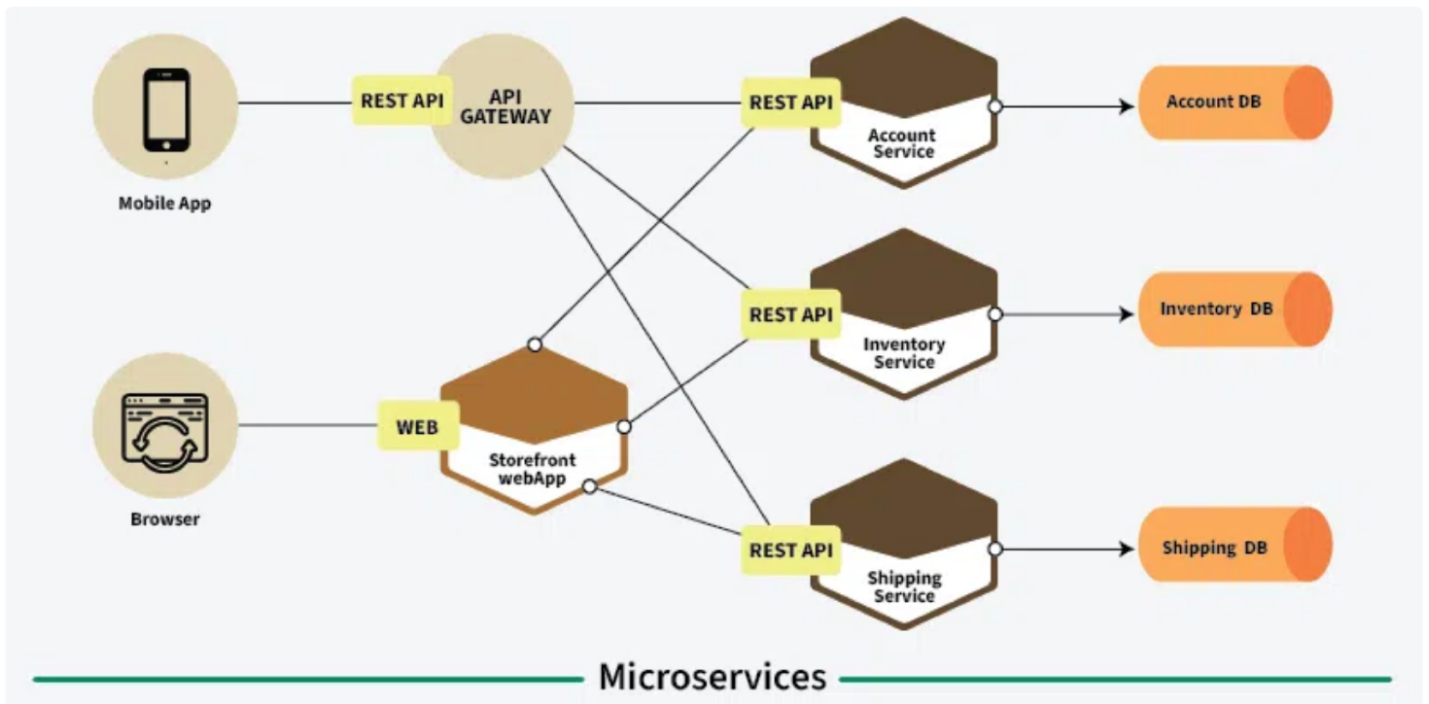
Microservices are an architectural approach to developing software applications as a collection of small, independent services that communicate with each other over a network.

- Instead of building a monolithic application where all the functionality is tightly integrated into a single codebase, microservices break down the application into smaller, loosely coupled services.
- Can be written in a variety of programming languages, and frameworks, and each service acts as a mini-application on its own.
- A small, loosely coupled service that is designed to perform a specific business function and each microservice can be developed, deployed, and scaled independently.



How Do Micro Services Work

- Each microservice handles a particular business feature, like user authentication or product management, allowing for specialized development.
- Services interact via APIs, facilitating standardized information exchange and integration.
- Different technologies can be used for each service, enabling teams to select the best tools for their needs.
- Microservices can be updated independently, reducing risks during changes and enhancing system resilience.



Main components of Microservices Architecture

Main components of microservices architecture include:

- **Microservices:** Small, loosely coupled services that handle specific business functions, each focusing on a distinct capability.
- **API Gateway:** Acts as a central entry point for external clients also they manage requests, authentication and route the requests to the appropriate microservice.
- **Service Registry and Discovery:** Keeps track of the locations and addresses of all microservices, enabling them to locate and communicate with each other dynamically.
- **Load Balancer:** Distributes incoming traffic across multiple service instances and prevent any of the microservice from being overwhelmed.
- **Containerization:** Docker encapsulate microservices and their dependencies and orchestration tools like Kubernetes manage their deployment and scaling.
- **Event Bus/Message Broker:** Facilitates communication between microservices, allowing pub/sub asynchronous interaction of events between components/microservices.
- **Database per Microservice:** Each microservice usually has its own database, promoting data autonomy and allowing for independent management and scaling.
- **Caching:** Cache stores frequently accessed data close to the microservice which improved performance by reducing the repetitive queries.
- **Fault Tolerance and Resilience Components:** Components like circuit breakers and retry mechanisms ensure that the system can handle failures gracefully, maintaining overall functionality.